Name: ABDUL REHMAN SAEED
Reg. No.: FA22-BCS-055
Assignment No.: 01
Course Title: Machine Learning

GitHub Repository

Source code and notebook are available here:
https://github.com/AbdulRehman393/machine_learning_coursework

Linear Regression and Logistic Regression Implementation (Python)

Objective

This notebook implements:

1. **Linear Regression** to predict a continuous target value (house price).
2. **Logistic Regression** to perform binary classification (cancer diagnosis).

The models are implemented using Python in Google Colab, including dataset loading, preprocessing, training, and evaluation.

Linear Regression (House Price Prediction)

Dataset for Linear Regression (House Prices)

- **Source:** Kaggle — *House Prices: Advanced Regression Techniques* (`train.csv`)
- **Target variable:** `SalePrice` (continuous)
- **Problem type:** Regression

The goal is to learn a relationship between selected house features and the final sale price.

Note: `train.csv` is loaded from Google Drive in this Colab notebook.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call dr
```

```
import pandas as pd

df = pd.read_csv("/content/drive/MyDrive/train.csv")
print(df.shape)     # rows, columns
df.head()
```

```
(1460, 81)
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	Land(
0	1	60	RL	65.0	8450	Pave	NaN	Reg	
1	2	20	RL	80.0	9600	Pave	NaN	Reg	
2	3	60	RL	68.0	11250	Pave	NaN	IR1	
3	4	70	RL	60.0	9550	Pave	NaN	IR1	
4	5	60	RL	84.0	14260	Pave	NaN	IR1	

5 rows × 81 columns

Feature Selection (Input Variables)

The full dataset contains many columns (81). For a simple beginner-friendly regression model, four numeric features are selected:

- `GrLivArea`: Above ground living area (in square feet)
- `BedroomAbvGr`: Number of bedrooms above ground
- `FullBath`: Number of full bathrooms
- `OverallQual`: Overall material and finish quality (higher is better)

These features form the input matrix **X**, and `SalePrice` is the output variable **y**.

```
features = ["GrLivArea", "BedroomAbvGr", "FullBath", "OverallQual"]
target = "SalePrice"

df_small = df[features + [target]].dropna()   # remove missing rows
X = df_small[features]
y = df_small[target]

print("X shape:", X.shape)
print("y shape:", y.shape)
df_small.head()
```

```
X shape: (1460, 4)
y shape: (1460,)
```

	GrLivArea	BedroomAbvGr	FullBath	OverallQual	SalePrice
0	1710	3	2	7	208500
1	1262	3	2	6	181500
2	1786	3	2	7	223500
3	1717	3	1	7	140000
4	2198	4	2	8	250000

Next steps: Generate code with `df_small` New interactive sheet

Train-Test Split

The dataset is divided into:

- **Training set (80%)**: used to train the model
- **Test set (20%)**: used to evaluate the model on unseen data

This helps check whether the model generalizes well instead of memorizing training data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train size:", X_train.shape)
print("Test size :", X_test.shape)
```

```
Train size: (1168, 4)
Test size : (292, 4)
```

Feature Scaling (Standardization)

Before training, the input features are standardized using **StandardScaler** so that each feature has:

- mean ≈ 0
- standard deviation ≈ 1

Scaling improves numerical stability and helps machine learning models train more effectively.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Model Training (Linear Regression)

Linear Regression learns a linear relationship between input features and the target variable. The model aims to minimize the prediction error between actual and predicted house prices.

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

Evaluation Metrics (Regression)

The regression model is evaluated using:

- **MAE (Mean Absolute Error):** average absolute difference between actual and predicted prices
- **RMSE (Root Mean Squared Error):** penalizes large errors more strongly than MAE
- **R^2 Score:** proportion of variance explained by the model (closer to 1 indicates better performance)

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

y_pred = lin_reg.predict(X_test_scaled)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Linear Regression (House Price Prediction)")
print("MAE :", mae)
print("RMSE:", rmse)
print("R2  :", r2)
```

```
Linear Regression (House Price Prediction)
MAE : 28290.357044877805
RMSE: 42813.43371836445
R2  : 0.7610284025418284
```

Logistic Regression using Breast Cancer dataset

Dataset for Logistic Regression (Breast Cancer)

The House Prices dataset is a regression dataset (continuous target), so it is not suitable for Logistic Regression.

Therefore, Logistic Regression is implemented using the **Breast Cancer Wisconsin dataset** from `sklearn`, which is a **binary classification** dataset:

- `malignant`
- `benign`

Problem type: Binary Classification

Model Training (Logistic Regression)

Logistic Regression is a classification algorithm that predicts the probability of a class using a sigmoid function. The predicted probability is then converted into a class label (malignant/benign).

Evaluation Metrics (Classification)

- **Accuracy:** overall proportion of correct predictions
- **Confusion Matrix:** counts of correct/incorrect predictions per class
- **Precision, Recall, F1-score:** class-wise performance measures

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_re

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target)  # 0=malignant, 1=benign

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_scaled, y_train)

y_pred = log_reg.predict(X_test_scaled)

print("Logistic Regression (Breast Cancer Classification)")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, targe
```

```
Logistic Regression (Breast Cancer Classification)
Accuracy: 0.9824561403508771

Confusion Matrix:
 [[41  1]
 [ 1 71]]

Classification Report:
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

Conclusion

Linear Regression was successfully implemented to predict house prices using selected numerical features, and performance was evaluated using MAE, RMSE, and R^2.

Logistic Regression was successfully implemented for binary classification on the breast cancer dataset, and performance was evaluated using accuracy, confusion matrix, and classification report.

This notebook demonstrates a complete supervised learning workflow: dataset loading, preprocessing, model training, and evaluation.